

Árvores geradoras mínimas

Grafos e Redes

Luiz Gustavo Cordeiro

Ilhas & Pontes

Problema

Um arquipélago possui várias ilhas. Podemos construir uma ponte entre qualquer par de ilhas, e o custo da ponte é a distância entre elas.

Queremos conectar todas as ilhas com o menor custo total.

Vértice: uma ilha.

Aresta: uma ponte que pode ser construída.

Peso: custo de construção da ponte.

Solução: conjunto de pontes que conecta todas as ilhas.

No jogo

Use *Construir todas as pontes* para exibir o grafo completo e os pesos de suas arestas.

Árvore geradora

Seja $G = (V, E)$ um grafo conexo e não dirigido.

Definição

Uma **árvore geradora** de G é um subgrafo $T = (V, F)$ que contém todos os vértices de G , é conexo e não possui ciclos.

Toda árvore geradora possui exatamente $|F| = |V| - 1$ arestas.

Caso não ponderado

Se todas as arestas têm o mesmo peso, qualquer árvore geradora serve. Uma DFS ou BFS produz uma delas.

Árvore geradora mínima

Considere agora um peso $p(e)$ associado a cada aresta $e \in E$. O peso de uma árvore geradora $T = (V, F)$ é $p(T) = \sum_{e \in F} p(e)$.

Definição

Uma **árvore geradora mínima** (AGM) é uma árvore geradora T^* tal que $p(T^*) \leq p(T)$ para toda árvore geradora T de G .

Uma AGM pode não ser única. Se G não for conexo, obtemos uma **floresta geradora mínima**, com uma AGM por componente.

Aplicações

Infraestrutura: ligar locais com cabos, estradas, tubulações ou linhas de transmissão, minimizando o custo de construção.

Redes: construir uma estrutura básica de comunicação que alcance todos os equipamentos com custo total mínimo.

Agrupamento: remover as arestas mais pesadas de uma AGM separa os dados em grupos próximos.

Labirintos: uma árvore geradora das células cria um labirinto com exatamente um caminho entre cada par de células.

Referência principal

Sedgewick, *Algorithms in C, Part 5: Graph Algorithms*, Capítulo 20.

Algoritmo de Prim

Prim mantém uma única árvore. Em cada passo, incorpora o vértice externo ligado a ela pela aresta mais leve disponível.

```
PRIM(G, peso, raiz):
  para cada vertice v de G:
    dist_a_arvore[v] = INFINITO
    pai[v] = NULO
  dist_a_arvore[raiz] = 0
  Q = lista com os vertices de V
  // Q normalmente e implementada como uma min-heap
  // ordenada de acordo com dist_a_arvore
  enquanto Q nao estiver vazia:
    u = EXTRAIR_MINIMO(Q, dist_a_arvore)
    para cada aresta {u, v}:
      se v pertence a Q e
        peso(u,v) < dist_a_arvore[v]:
          pai[v] = u
          dist_a_arvore[v] = peso(u,v)
  devolva {{pai[v], v} : v != raiz}
```

Custo

$O(E \log V)$ com lista de adjacências e heap binário. Para grafos densos, uma implementação simples em $O(V^2)$ pode ser conveniente.

Estrutura de conjuntos disjuntos

Partição e representantes

Uma estrutura de *conjuntos disjuntos* codifica uma partição de V . Para cada bloco, escolhemos um de seus elementos como **representante**. Dois elementos u e v estão no mesmo bloco exatamente quando $\text{REPRESENTANTE}(u) = \text{REPRESENTANTE}(v)$.

Representamos cada bloco por uma árvore cuja raiz é o representante.

```
CRIAR_CONJUNTO(v):  
    pai[v] = v  
REPRESENTANTE(v):  
    se pai[v] == v: devolva v  
    devolva REPRESENTANTE(pai[v])  
UNIR(u, v):  
    ru = REPRESENTANTE(u)  
    rv = REPRESENTANTE(v)  
    se ru != rv:  
        pai[rv] = ru
```

Operações

$\text{CRIAR_CONJUNTO}(v)$ cria $\{v\}$. $\text{UNIR}(u, v)$ liga as raízes. União por tamanho e compressão de caminhos dão custo amortizado quase constante.

Algoritmo de Kruskal

Kruskal começa com uma floresta de vértices isolados. Examina as arestas em ordem crescente de peso e aceita cada uma que liga componentes distintas.

```
KRUSKAL(G, peso):  
  T = { }  
  para cada vertice v de G:  
    CRIAR_CONJUNTO(v)  
  arestas = ORDENAR(E, por peso crescente)  
  para cada aresta {u, v} em arestas:  
    se REPRESENTANTE(u) != REPRESENTANTE(v):  
      T = T uniao {{u, v}}  
      UNIR(u, v)  
      se |T| == |V| - 1:  
        interrompa  
  devolva T
```

Custo

$O(E \log E)$, dominado pela ordenação. Os conjuntos disjuntos permitem testar e unir componentes com baixo custo.